

Linux Essentials for DevOps

Filesystem, Users, Permissions, Processes & systemd

Linux knowledge becomes especially important in DevOps because tools such as Docker, Jenkins, Nginx, databases, application servers, and cloud workloads commonly run on Linux systems.

You do not need to become a Linux administrator before learning DevOps.

The goal is to understand the parts of Linux that directly affect:

- Application deployment
- File permissions
- Service management
- Process troubleshooting
- Logs
- Configuration
- Package installation
- Docker and CI/CD tools

1. Understanding `/usr`

The name `/usr` can be confusing because it looks like it means:

```
user
```

But:

```
/usr ≠ user home directories
```

User home directories normally live inside:

```
/home
```

For example:

```
/home/aditya  
/home/ubuntu
```

`/usr` mostly contains installed programs, libraries, and related system resources.

Common directories include:

```
/usr/bin  
/usr/lib  
/usr/local
```

A useful DevOps mental model is:

```
/usr  
≈ installed system software and related resources
```

You do not need to memorize every directory inside `/usr`.

2. `/usr/bin`

`/usr/bin` contains many command executables.

For example:

```
/usr/bin/java  
/usr/bin/nodejs  
/usr/bin/curl  
/usr/bin/git
```

A command is often simply a program stored somewhere in the filesystem.

When you type:

```
curl
```

the shell may actually execute:

```
/usr/bin/curl
```

The shell saves you from entering the complete path every time.

You can inspect where a command is located using:

```
which curl
```

For example:

```
/usr/bin/curl
```

3. `/usr/local`

`/usr/local` commonly contains software and resources installed locally by the system administrator rather than software managed directly by the operating system's normal package system.

For DevOps fundamentals, remember:

```
/usr/local  
→ locally installed software/resources
```

Deep knowledge of its internal structure is not required initially.

4. Linux Filesystem Interfaces

Not everything visible inside the Linux filesystem is an ordinary file stored on your disk.

Linux also exposes devices and system information through filesystem-like interfaces.

Two important examples are:

```
/dev  
/proc
```

A useful Linux mental model is:

| Linux exposes many system resources through filesystem-like interfaces.

5. `/dev` — Devices

`/dev` stands for:

```
devices
```

Consider a normal file:

```
/home/aditya/notes.txt
```

That file stores data somewhere on disk.

Now consider a physical disk connected to the machine. Linux may expose that device using something like:

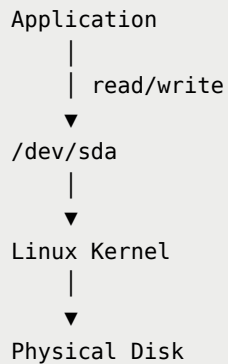
```
/dev/sda
```

Do not think of `/dev/sda` as a file containing the disk.

Instead, think of it as:

```
a filesystem-style interface through which programs  
can communicate with the disk
```

Conceptually:



Other examples include:

```
/dev/null  
/dev/tty
```

For DevOps fundamentals:

```
/dev  
≈ devices exposed through filesystem interfaces
```

You do not need to learn device management in depth yet.

6. Why Linux Represents Devices This Way

Programs already know how to perform operations such as:

```
open  
read  
write  
close
```

Linux provides filesystem-like interfaces for many devices so programs can interact with different system resources using familiar operations.

This relates to the famous Unix idea:

| Everything is a file.

This is slightly simplified, but it is a useful beginner mental model.

7. `/dev/null`

`/dev/null` is a special device that discards anything written to it.

Think of it as the:

black hole of Linux

For example:

```
echo "Hello DevOps"
```

Output:

```
Hello DevOps
```

If the output is redirected to `/dev/null`:

```
echo "Hello DevOps" > /dev/null
```

nothing appears.

Conceptually:

```
Command Output
  ↓
/dev/null
  ↓
discarded
```

This becomes useful when working with:

- Scripts
- Cron jobs
- Logs
- Background commands
- Command output redirection

8. `/proc` — Process and Kernel Information

`/proc` is different from ordinary directories.

It is a:

Virtual filesystem

The Linux kernel exposes information about running processes and the system through `/proc`.

Suppose a running process has:

```
PID = 1234
```

Linux may expose information about it under:

```
/proc/1234
```

Conceptually:

```
Running Process
  ↓
Linux Kernel
  ↓
exposes information
  ↓
/proc/1234
```

The files inside `/proc` are not simply normal files permanently stored on disk.

They expose live information generated by the kernel.

Useful comparison:

```
/opt
→ application files

/etc
→ configuration files

/var/log
→ log files

/dev
→ interfaces to devices

/proc
→ live process/kernel information
```

9. Users and Groups

Imagine one Linux server running:

```
Spring Boot application
MySQL
Nginx
Jenkins
Docker
```

Humans may also log into the same server:

```
Developer
Administrator
DevOps Engineer
```

Every person and application should not have unrestricted access to everything.

Otherwise situations like these could occur:


```
Jenkins → deletes MySQL data  
Nginx → modifies SSH configuration  
Developer → reads private SSH keys  
Application → modifies critical system files
```

Linux therefore needs a way to identify **who is performing an operation**.

10. Linux Is a Multi-User Operating System

Linux is designed around the idea that multiple users and programs may operate on the same machine.

When something tries to access a resource, Linux effectively needs to determine:

```
Who are you?  
↓  
What are you allowed to do?
```

For example:

```
Request:  
"Open this file"  
  
Linux:  
"Who are you?"  
  
User:  
"ubuntu"  
  
Linux:  
"Does ubuntu have permission to open this file?"
```

That identity is represented by a **Linux user**.

11. Linux Users Are Not Only Humans

When beginners hear the word:

```
user
```

they usually think about humans.

But Linux users can also represent services and applications.

Examples:

```
ubuntu  
root  
nginx  
mysql  
jenkins  
www-data
```

For example:

```
Nginx process  
  ↓  
runs as  
  ↓  
www-data user
```

MySQL may run as:

```
mysqld process  
  ↓  
runs as  
  ↓  
mysql user
```

Jenkins may run as:

```
Jenkins process  
  ↓  
runs as
```

```
↓  
jenkins user
```

Linux can then restrict what each service is allowed to access.

For example:

```
jenkins  
→ Jenkins workspace  
  
mysql  
→ database files  
  
nginx  
→ web content
```

This is much safer than:

```
everything runs as root
```

12. Checking the Current User

To find your current Linux user:

```
whoami
```

Example:

```
ubuntu
```

Commands you execute normally run with the permissions associated with your current user.

13. UID and GID

Linux internally associates users with numeric IDs.

A user has a:

```
UID
```

which means:

```
User ID
```

Run:

```
id
```

You may see:

```
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),27(sudo)
```

Break it down:

```
uid=1000(ubuntu)
```

means:

```
Username → ubuntu  
UID      → 1000
```

Then:

```
gid=1000(ubuntu)
```

represents the user's primary group.

```
GID
=
Group ID
```

Finally:

```
groups=...
```

shows the groups that the user belongs to.

14. Why Users and Groups Matter in DevOps

Suppose your application tries to create:

```
/var/log/myapp/app.log
```

but fails with:

```
Permission denied
```

The problem may not be inside your application code.

Maybe your application runs as:

```
myapp
```

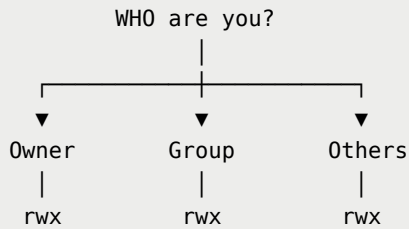
but the `myapp` Linux user does not have permission to write inside:

```
/var/log/myapp
```

This is why Linux permissions are directly relevant when deploying applications.

15. Linux File Permissions

Every file has three important permission categories:



The permission characters mean:

```
r → read
w → write
x → execute
```

Suppose:

```
-rw-r----- aditya developers secret.txt
```

Break it into:

```
- | rw- | r-- | ---
  ↑   ↑   ↑
  owner group others
```

The file has:

```
Owner → aditya
Group → developers
```

Permissions:

```
aditya
→ read + write
```

```
developers
→ read only

others
→ no permissions
```

16. How Linux Chooses Which Permission Applies

Suppose the current user is:

```
rohit
```

and Rohit belongs to:

```
developers
devops
```

The file is:

```
-rw-r----- aditya developers secret.txt
```

Linux checks:

```
Is rohit the owner?
  ↓
No

Is rohit part of developers?
  ↓
Yes

Use group permissions
  ↓
r--
```

Therefore Rohit can:

Read	✓
Modify	✗
Execute	✗

Now suppose another user:

guest

is not the owner and is not part of `developers`.

Linux uses:

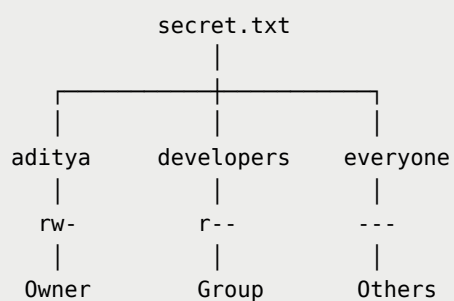
others permissions

which are:

Therefore:

guest → no access

The overall model is:



17. `ls` vs `ls -l`

A simple:

```
ls
```

shows filenames.

Example:

```
app.jar  
deploy.sh  
notes.txt
```

A long listing:

```
ls -l
```

shows additional information, including permissions and ownership.

For example:

```
-rw-r--r-- ubuntu ubuntu deploy.sh
```

18. Shell Scripts and Execute Permission

A `.sh` file is a shell script.

It is essentially a text file containing commands that Linux can execute.

Example:

```
nano deploy.sh
```

Content:

```
echo "Deploying Spring Boot application..."
```

Check permissions:

```
ls -l deploy.sh
```

Suppose you see:

```
-rw-r--r-- deploy.sh
```

Permissions:

```
Owner   → rw-  
Group   → r--  
Others  → r--
```

There is no:

```
x
```

So attempting:

```
./deploy.sh
```

may produce:

```
Permission denied
```

because the file does not have execute permission.

19. **chmod**

chmod means:

change mode

Practically:

| Change the permissions of a file or directory.

20. Symbolic `chmod`

A readable form is:

```
chmod u+x deploy.sh
```

Break it down:

```
u → user/owner  
+ → add  
x → execute
```

Therefore:

```
chmod u+x deploy.sh
```

means:

| Add execute permission for the owner.

Before:

```
-rw-r--r--
```

After:

```
-rwxr--r--
```

21. Permission Targets

With symbolic `chmod` :

```
u → user/owner  
g → group  
o → others  
a → all
```

Operations:

```
+ → add permission  
- → remove permission  
= → set permissions exactly
```

Permissions:

```
r → read  
w → write  
x → execute
```

22. Adding Group Write Permission

Suppose:

```
-rw-r--r--
```

Run:

```
chmod g+w shared.txt
```

Meaning:

```
g → group  
+ → add
```

```
w → write
```

Result:

```
-rw-rw-r--
```

Now:

```
Owner  → read + write  
Group  → read + write  
Others → read
```

This is useful when members of a group need to modify a shared file.

23. Removing Permission from Others

Suppose:

```
-rw-r--r--
```

Run:

```
chmod o-r secret.txt
```

Meaning:

```
o → others  
- → remove  
r → read
```

Result:

```
-rw-r-----
```

Now users outside the owner/group cannot read the file.

24. Combining Permissions

You can modify multiple permissions together.

Add read and write to owner

```
chmod u+rw file.txt
```

Add read and execute to group

```
chmod g+rx deploy.sh
```

Remove everything from others

```
chmod o-rwx secret.txt
```

25. Using `=` with `chmod`

Consider:

```
chmod u=rw secret.txt
```

This does not mean:

```
add read and write
```

It means:

| Set the owner's permissions to exactly read and write.

If the owner previously had:

```
rwX
```

it becomes:

```
rw-
```

because `x` was not included.

Remember:

```
+ → add  
- → remove  
= → replace with exactly these permissions
```

26. Numeric Permissions

Linux permissions can also be represented numerically.

The values are:

```
read    = 4  
write   = 2  
execute = 1
```

Combine them by addition.

```
rwX
```

```
4 + 2 + 1 = 7
```

```
rw-
```

```
4 + 2 = 6
```

r-x

4 + 1 = 5

r--

4

27. Understanding **chmod 755**

Consider:

```
chmod 755 deploy.sh
```

Break it into:

7		5		5
↑		↑		↑
u		g		o

Convert each number:

```
7 → rwx
5 → r-x
5 → r-x
```

Therefore:

```
rwXr-xr-x
```

Meaning:


```
Owner  → read + write + execute
Group  → read + execute
Others → read + execute
```

28. Avoid Blindly Using `chmod 777`

A common bad debugging approach is:

```
Permission denied
  ↓
chmod 777
```

But:

```
chmod 777 file
```

means:

```
Owner  → rwx
Group  → rwx
Others → rwx
```

Everyone can:

- Read
- Modify
- Execute

It may remove the immediate permission error while creating a security problem.

A better question is:

| Which user actually needs which permission?

That mindset is more important than memorizing `chmod` commands.

29. Ownership vs Permissions

Sometimes the permission bits are reasonable, but the wrong user owns the file.

Example:

```
-rw----- root root application.properties
```

Suppose the application runs as:

```
myapp
```

`myapp` cannot read the file.

A bad solution might be:

```
chmod 777 application.properties
```

But perhaps the real problem is:

| The file should belong to the application user or group.

This is where `chown` is useful.

30. `chown`

`chown` means:

```
change owner
```

Example:

```
sudo chown ubuntu app.jar
```

This changes the owner of `app.jar` to:

```
ubuntu
```

31. Changing Owner and Group Together

Common syntax:

```
sudo chown ubuntu:developers app.jar
```

Meaning:

```
Owner → ubuntu  
Group → developers
```

After:

```
ls -l app.jar
```

you may see:

```
-rw-r--r-- ubuntu developers app.jar
```

32. Recursive Ownership

Suppose your application directory looks like:

```
/opt/myapp/  
├─ app.jar  
├─ config/  
├─ scripts/  
└─ data/
```

If the entire directory should belong to the `myapp` user:

```
sudo chown -R myapp:myapp /opt/myapp
```

- **R** means:

```
recursive
```

The ownership change applies to the directory and its contents.

Be careful with recursive commands because they can affect a large number of files.

33. **chgrp**

You can change only the group using:

```
chgrp developers file.txt
```

However, knowing:

```
chown owner:group
```

is enough for most beginner DevOps situations.

34. Root User

Linux needs an administrative identity that can manage the entire system.

That user is:

```
root
```

root is the Linux superuser.

Conceptually:

```
Normal user
  ↓
restricted by permissions

Root
  ↓
administrative authority over almost the entire system
```

Root normally has:

```
UID = 0
```

Its special privileges are fundamentally associated with UID `0`.

35. Why We Do Not Work as Root All the Time

Working with unlimited permissions makes mistakes much more dangerous.

For example, one incorrect destructive command could damage important system files.

There is also a security concern:

```
Application compromised
  ↓
Application runs as root
  ↓
Attacker potentially gains enormous system access
```

Good Linux practice therefore follows the:

| Principle of Least Privilege

Give a user or process only the permissions required to perform its job.

36. `sudo`

A normal user sometimes needs to perform administrative operations.

That is where:

```
sudo
```

is used.

Think of `sudo` as:

Execute this particular command with elevated privileges, provided the user is authorized.

For example:

```
apt install nginx
```

may fail for a normal user.

Instead:

```
sudo apt install nginx
```

can perform the privileged package installation.

37. Normal by Default, Privileged When Necessary

Most everyday commands can run normally:

```
ls  
cd  
cat  
java  
git
```

When administrative permission is actually required:

```
sudo apt install nginx
```

or:

```
sudo systemctl restart nginx
```

The model becomes:

```
normal privilege by default
      ↓
elevated privilege only when required
```

This is safer than constantly working as `root`.

38. Do Not Use `sudo` for Every Permission Error

Suppose:

```
mkdir /opt/myapp
```

returns:

```
Permission denied
```

Since `/opt` is a system-level location, this may legitimately require:

```
sudo mkdir /opt/myapp
```

But do not conclude:

```
Permission denied
      ↓
```

```
always use sudo
```

Instead ask:

```
Why am I denied?
```

Possible reasons:

```
Wrong owner?  
Wrong permissions?  
System directory?  
Application should use another location?  
Command genuinely requires administrative privileges?
```

Understand the reason before elevating privileges.

39. Program vs Process

A program stored on disk is not the same thing as a running process.

Consider:

```
app.jar
```

The JAR file exists on disk even when the application is not running.

When you execute:

```
java -jar app.jar
```

Linux starts a running instance.

Conceptually:

```
PROGRAM  
↓  
instructions stored somewhere
```



```
PROCESS
↓
running instance of those instructions
```

For a Java application:

```
Shell
↓
find java executable
↓
ask Linux to execute it
↓
Linux creates a process
↓
JVM starts
↓
JVM loads app.jar
↓
Spring Boot application runs
```

40. One Program Can Create Multiple Processes

Suppose:

```
app.jar
```

exists once on disk.

You could execute:

```
java -jar app.jar
```

multiple times.

Ignoring port conflicts, Linux could create:

```
Process 1 → PID 5210
Process 2 → PID 5287
Process 3 → PID 5312
```

Same program.

Different running processes.

So:

```
Program
→ static

Process
→ dynamic
```

A process has runtime information such as:

```
PID
memory
CPU usage
environment variables
open files
network sockets
user
current working directory
process state
```

41. Who Manages Processes?

The Linux **kernel** manages processes.

When a program starts, the kernel tracks information about the resulting process.

Conceptually:

```
Kernel

PID 1001
├─ command: java
└─ user: ubuntu
```

```
|— memory: ...  
|— CPU state: ...  
|— open files: ...  
└— parent: 900
```

```
PID 1002  
|— command: nginx  
|— user: www-data  
└— ...
```

42. PID — Process ID

Every running process receives a:

```
PID
```

which means:

```
Process ID
```

The PID allows Linux and users to distinguish between many running processes.

Example:

```
java → PID 4821  
mysql → PID 830  
sshd → PID 701
```

44. The Shell Is Also a Process

Bash itself is a program.

When Bash is running, it is also a process.

When you execute:

ls

a simplified flow is:

```
bash process
  ↓
launch ls process
  ↓
wait for ls
  ↓
ls exits
  ↓
bash shows prompt again
```

This helps explain foreground and background processes.

51. ps

ps means:

process status

Running:

ps

shows processes associated with the current session.

Even **ps** itself temporarily becomes a process while it is running.

What Is a Service?

A **service** is a program or application intended to provide functionality continuously, usually without direct interactive terminal usage.

Examples:

```
SSH server
Nginx
MySQL
Docker daemon
Jenkins
Spring Boot backend
```

A short command may behave like:

```
start
↓
perform task
↓
exit
```

A service usually behaves more like:

```
start
↓
keep running
↓
serve requests
↓
keep running
↓
machine shuts down
```

61. Service vs Process

A service is not separate from processes.

Ultimately, a service runs through one or more processes.

A useful distinction is:

```
Process
→ running instance of a program
```

Service

→ long-running functionality that is managed as an application

Service Manager

We want something above the application process:

Service Manager

↓

Java Application

The manager should know:

- How to start the application
- How to stop it
- How to restart it
- Which user runs it
- Which directory it runs from
- What environment variables it receives
- Whether it starts after boot
- Whether it restarts after failure
- Where logs go
- What other services it depends on

On many modern Linux distributions, this role is handled by:

systemd

Ubuntu uses `systemd`.

What Is `systemd` ?

`systemd` is a system and service manager used by many Linux distributions.

On a typical systemd-based machine, it becomes one of the earliest important user-space processes started after the kernel.

It normally runs as:

```
PID 1
```

You can inspect it using:

```
ps -p 1
```

You may see something like:

PID	TTY	TIME	CMD
1	?	00:00	systemd

This gives a useful architecture:

```
Linux Kernel
  ↓
systemd
PID 1
  ↓
manages services
  ↓
nginx / docker / mysql / application
```

Complete Mental Model

The Linux concepts covered here connect together.

```
Filesystem
  ↓
Where applications, configuration and logs live

Users & Groups
  ↓
Who is performing an operation?

Permissions
  ↓
```

What is that user allowed to do?

Program

↓

Static instructions stored on disk

Process

↓

Running instance of a program

PID

↓

Identity of that running process

Service

↓

Long-running application functionality

systemd

↓

Manages long-running services